

```
In [1]: import pandas as pd
df = pd.read_csv('nba.csv')
```

```
In [2]: df.head()
```

Out[2]:

|   | Unnamed: 0 | name            | gp | min  | pts | fgm | fga | fg   | 3p_made | 3pa | ... | fta | ft   | oreb |
|---|------------|-----------------|----|------|-----|-----|-----|------|---------|-----|-----|-----|------|------|
| 0 | 0          | Brandon Ingram  | 36 | 27.4 | 7.4 | 2.6 | 7.6 | 34.7 | 0.5     | 2.1 | ... | 2.3 | 69.9 | 0.7  |
| 1 | 1          | Andrew Harrison | 35 | 26.9 | 7.2 | 2.0 | 6.7 | 29.6 | 0.7     | 2.8 | ... | 3.4 | 76.5 | 0.5  |
| 2 | 2          | JaKarr Sampson  | 74 | 15.3 | 5.2 | 2.0 | 4.7 | 42.2 | 0.4     | 1.7 | ... | 1.3 | 67.0 | 0.5  |
| 3 | 3          | Malik Sealy     | 58 | 11.6 | 5.7 | 2.3 | 5.5 | 42.6 | 0.1     | 0.5 | ... | 1.3 | 68.9 | 1.0  |
| 4 | 4          | Matt Geiger     | 48 | 11.5 | 4.5 | 1.6 | 3.0 | 52.4 | 0.0     | 0.1 | ... | 1.9 | 67.4 | 1.0  |

5 rows × 22 columns

```
In [3]: print("Value count for 'target_5yrs' :")
display(df['target_5yrs'].value_counts())

print("\nData type of 'target_5yrs' :")
display(df['target_5yrs'].dtype)
```

Value count for 'target\_5yrs' :

```
target_5yrs
1      831
0      509
Name: count, dtype: int64
```

Data type of 'target\_5yrs' :

```
dtype('int64')
```

```
In [4]: columns_to_drop = ['Unnamed: 0', 'name']
df_cleaned = df.drop(columns=columns_to_drop)

print(f"Original number of columns: {df.shape[1]}")
print(f"Number of columns after dropping: {df_cleaned.shape[1]}")

print("\nFirst 5 rows of the DataFrame after dropping non-predictive columns:")
display(df_cleaned.head())
```

Original number of columns: 22

Number of columns after dropping: 20

First 5 rows of the DataFrame after dropping non-predictive columns:

|   | gp | min  | pts | fgm | fga | fg   | 3p_made | 3pa | 3p   | ftm | fta | ft   | oreb | dreb | reb | as  |
|---|----|------|-----|-----|-----|------|---------|-----|------|-----|-----|------|------|------|-----|-----|
| 0 | 36 | 27.4 | 7.4 | 2.6 | 7.6 | 34.7 | 0.5     | 2.1 | 25.0 | 1.6 | 2.3 | 69.9 | 0.7  | 3.4  | 4.1 | 1.1 |
| 1 | 35 | 26.9 | 7.2 | 2.0 | 6.7 | 29.6 | 0.7     | 2.8 | 23.5 | 2.6 | 3.4 | 76.5 | 0.5  | 2.0  | 2.4 | 3.1 |
| 2 | 74 | 15.3 | 5.2 | 2.0 | 4.7 | 42.2 | 0.4     | 1.7 | 24.4 | 0.9 | 1.3 | 67.0 | 0.5  | 1.7  | 2.2 | 1.0 |
| 3 | 58 | 11.6 | 5.7 | 2.3 | 5.5 | 42.6 | 0.1     | 0.5 | 22.6 | 0.9 | 1.3 | 68.9 | 1.0  | 0.9  | 1.9 | 0.1 |
| 4 | 48 | 11.5 | 4.5 | 1.6 | 3.0 | 52.4 | 0.0     | 0.1 | 0.0  | 1.3 | 1.9 | 67.4 | 1.0  | 1.5  | 2.5 | 0.1 |

```
In [9]: correlation_matrix = df_cleaned.corr()

print("\nCorrelation of features with 'target_5yrs':")
display(correlation_matrix['target_5yrs'].sort_values(ascending=False))

high_corr_threshold = 0.8

highly_correlated_pairs = []

for i in range(len(correlation_matrix.columns)):
    for j in range(i + 1, len(correlation_matrix.columns)):
        col1 = correlation_matrix.columns[i]
        col2 = correlation_matrix.columns[j]

        if col1 != 'target_5yrs' and col2 != 'target_5yrs':
            if abs(correlation_matrix.loc[col1, col2]) > high_corr_threshold:
                highly_correlated_pairs.append((col1, col2, correlation_matrix.loc[col1, col2]))

print(f"\nHighly correlated features pairs (absolute correlation > {high_corr_threshold}):")
if highly_correlated_pairs:
    for col1, col2, corr_value in highly_correlated_pairs:
        print(f"- {col1} and {col2}: {corr_value:.2f}")
else:
    print("No highly correlated features pairs found above the threshold.")
```

Correlation of features with 'target\_5yrs':

|             |          |
|-------------|----------|
| target_5yrs | 1.000000 |
| gp          | 0.396833 |
| min         | 0.317805 |
| fgm         | 0.317594 |
| pts         | 0.315981 |
| reb         | 0.299406 |
| ftm         | 0.296841 |
| fta         | 0.296089 |
| oreb        | 0.293307 |

```
fga          0.292660
dreb         0.284677
tov          0.272348
stl          0.229811
fg           0.227134
blk          0.210114
ast          0.175353
ft           0.106706
3p_made      0.036619
3pa          0.018110
3p           -0.000004
Name: target_5yrs, dtype: float64
```

Highly correlated features pairs (absolute correlation > 0.8):

- min and pts: 0.91
- min and fgm: 0.90
- min and fga: 0.91
- min and tov: 0.83
- pts and fgm: 0.99
- pts and fga: 0.98
- pts and ftm: 0.90
- pts and fta: 0.88
- pts and tov: 0.85
- fgm and fga: 0.98
- fgm and ftm: 0.85
- fgm and fta: 0.84
- fgm and tov: 0.83
- fga and ftm: 0.83
- fga and fta: 0.81
- fga and tov: 0.85
- 3p\_made and 3pa: 0.98
- ftm and fta: 0.98
- ftm and tov: 0.80
- oreb and dreb: 0.84
- oreb and reb: 0.93
- dreb and reb: 0.98

```
In [10]: df_cleaned['points_per_minute'] = df_cleaned.apply(
          lambda row: row['pts'] / row['min'] if row['min'] > 0 else 0,
          axis=1
        )

print("First 5 rows of the DataFrame with the new ' points_per_minute' feature:")
display(df_cleaned.head())
```

First 5 rows of the DataFrame with the new ' points\_per\_minute' feature:

|   | gp | min  | pts | fgm | fga | fg   | 3p_made | 3pa | 3p   | ftm | ... | ft   | oreb | dreb | reb | ast |
|---|----|------|-----|-----|-----|------|---------|-----|------|-----|-----|------|------|------|-----|-----|
| 0 | 36 | 27.4 | 7.4 | 2.6 | 7.6 | 34.7 | 0.5     | 2.1 | 25.0 | 1.6 | ... | 69.9 | 0.7  | 3.4  | 4.1 | 1.9 |
| 1 | 35 | 26.9 | 7.2 | 2.0 | 6.7 | 29.6 | 0.7     | 2.8 | 23.5 | 2.6 | ... | 76.5 | 0.5  | 2.0  | 2.4 | 3.7 |
| 2 | 74 | 15.3 | 5.2 | 2.0 | 4.7 | 42.2 | 0.4     | 1.7 | 24.4 | 0.9 | ... | 67.0 | 0.5  | 1.7  | 2.2 | 1.0 |
| 3 | 58 | 11.6 | 5.7 | 2.3 | 5.5 | 42.6 | 0.1     | 0.5 | 22.6 | 0.9 | ... | 68.9 | 1.0  | 0.9  | 1.9 | 0.8 |
| 4 | 48 | 11.5 | 4.5 | 1.6 | 3.0 | 52.4 | 0.0     | 0.1 | 0.0  | 1.3 | ... | 67.4 | 1.0  | 1.5  | 2.5 | 0.3 |

5 rows × 21 columns

```
In [13]: print("Missing values before cleaning:")
display(df_cleaned.isnull().sum())

print("\nDataFrame Info:")
df_cleaned.info()
```

Missing values before cleaning:

```
gp          0
min          0
pts          0
fgm          0
fga          0
fg           0
3p_made      0
3pa          0
3p           0
ftm          0
fta          0
ft           0
oreb         0
dreb         0
reb          0
ast          0
stl          0
blk          0
tov          0
target_5yrs  0
points_per_minute  0
dtype: int64
```

DataFrame Info:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1340 entries, 0 to 1339
Data columns (total 21 columns):
#   Column                Non-Null Count  Dtype
---  -

```

|    |                   |               |         |
|----|-------------------|---------------|---------|
| 0  | gp                | 1340 non-null | int64   |
| 1  | min               | 1340 non-null | float64 |
| 2  | pts               | 1340 non-null | float64 |
| 3  | fgm               | 1340 non-null | float64 |
| 4  | fga               | 1340 non-null | float64 |
| 5  | fg                | 1340 non-null | float64 |
| 6  | 3p_made           | 1340 non-null | float64 |
| 7  | 3pa               | 1340 non-null | float64 |
| 8  | 3p                | 1340 non-null | float64 |
| 9  | ftm               | 1340 non-null | float64 |
| 10 | fta               | 1340 non-null | float64 |
| 11 | ft                | 1340 non-null | float64 |
| 12 | oreb              | 1340 non-null | float64 |
| 13 | dreb              | 1340 non-null | float64 |
| 14 | reb               | 1340 non-null | float64 |
| 15 | ast               | 1340 non-null | float64 |
| 16 | stl               | 1340 non-null | float64 |
| 17 | blk               | 1340 non-null | float64 |
| 18 | tov               | 1340 non-null | float64 |
| 19 | target_5yrs       | 1340 non-null | int64   |
| 20 | points_per_minute | 1340 non-null | float64 |

dtypes: float64(19), int64(2)  
memory usage: 220.0 KB



#### Summary of Feature Engineering Choices:

1. **Target Variable Definition:** We explicitly defined `target_5yrs` as the dependent variable for predicting player longevity. It's a binary variable (1 if a player played  $\geq 5$  years, 0 otherwise).
2. **Dropped Non-Predictive Columns:** We removed the `Unnamed: 0` (likely an index artifact) and `name` columns. `name` is a unique identifier and not predictive, removing it also prevents potential data leakage and reduces noise.
3. **Correlation Analysis:** We performed correlation analysis to understand feature relationships:
  - \* Identified features highly correlated with `target_5yrs` (e.g., `gp`, `min`, `pts`, `fgm`).
  - \* Detected significant multicollinearity among independent variables (e.g., `min` with `pts`, `fgm`, `fga`; `pts` with `fgm`, `fga`, `ftm`, `fta`; `oreb` with `dreb` and `reb`; `3p_made` with `3pa`). This suggests that some of these highly correlated features might be redundant, and a selection strategy might be needed in a modeling phase (e.g., keeping one, performing PCA, or creating a composite).
4. **Engineered Composite Feature ( `points_per_minute` ):** We created a new feature `points_per_minute` by dividing `pts` by `min`. This metric provides a normalized measure of scoring efficiency, accounting for playing time, and could be a more robust predictor than raw points or minutes alone.

In [ ]: